

架构辩论赛记录

成员A

Prompt

直接问"推荐什么架构"

AI 回答

方案A：分层架构 (Layered)

推荐理由：

维度	评价
10周可行性	极高
开发复杂度	较低
团队学习成本	低
可维护性	良好
性能	良好（本地调用，无网络开销）
扩展性	有限但足够应对初期业务

为什么不选其他方案？

- 方案B (MVC)：虽然开发速度最快，但业务扩大后控制器易膨胀、耦合度高，可维护性一般。
- 方案C (微服务)：开发复杂度高、学习成本高、10周内可行性极低，不适合小团队。
- 方案D (事件驱动)：异步调试复杂、可靠性要求高，短期内风险较大。

一句话总结：

分层架构在有限时间和资源下，风险最低、交付最可控，最适合4人团队在10周内完成稳定系统。

成员B

Prompt

提供详细项目约束后问,请设计一套适合4人团队、10周周期、可落地的分层架构实施方案，避免传统分层架构的缺点。

AI 回答

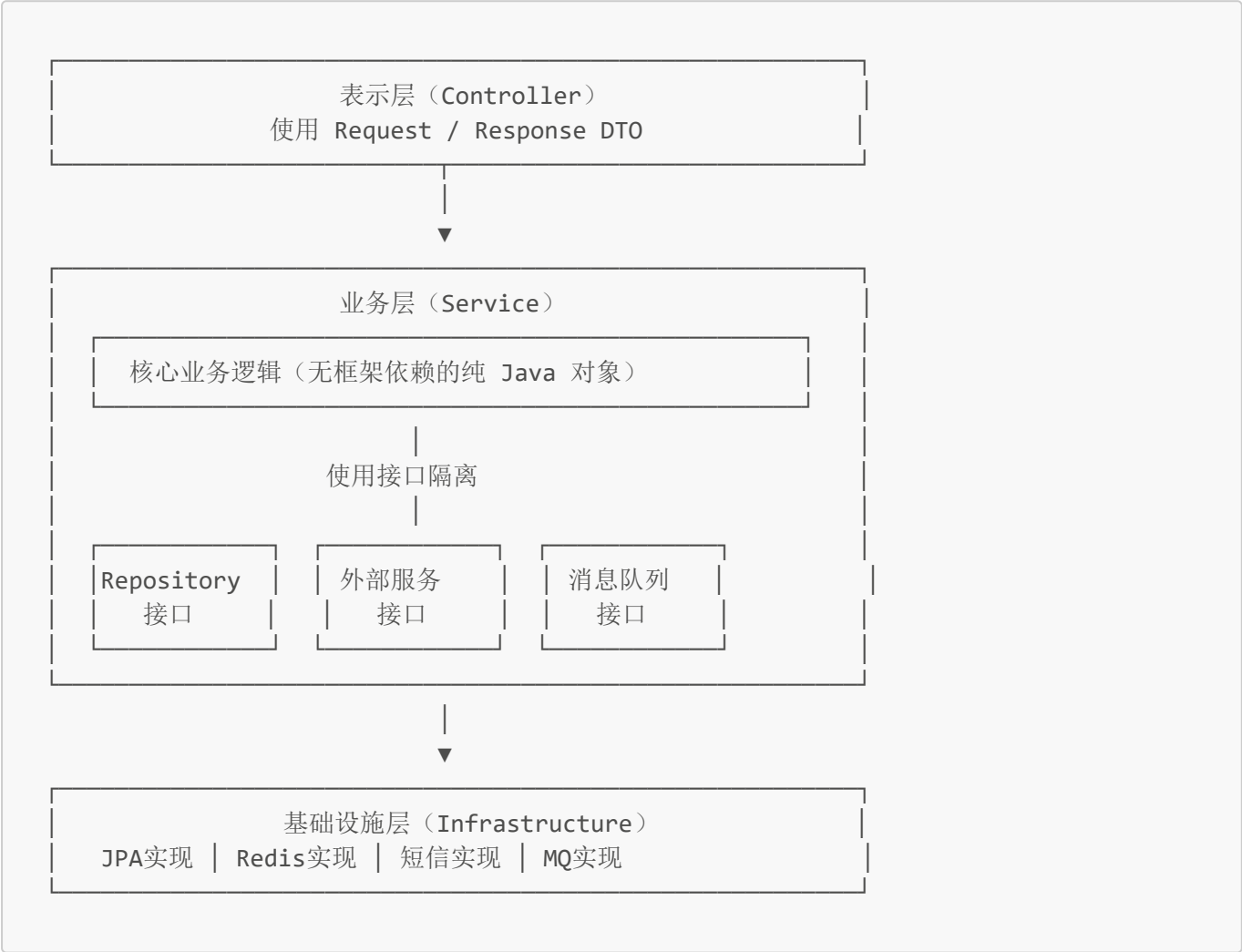
「简化版分层架构 + 整洁思想」混合模式详解

一、为什么要混合？

纯粹的分层架构容易导致业务逻辑泄漏（Controller 里写计算、Entity 里塞 SQL），而完整的整洁架构又太重（4 人小组无法承受）。混合模式的目标：用最低成本，解决分层架构最痛的三个问题：

- 业务逻辑与技术框架耦合
- 数据对象层层传递导致边界模糊
- 核心业务难以单独测试

二、混合模式的整体结构



关键特征：

- 保持分层架构的三层结构（表现层-业务层-数据层）
- 引入整洁架构的两个核心机制：实体层纯POJO、端口与适配器
- 放弃整洁架构的两个重机制：独立用例层、严格依赖倒置

三、核心落地要点（4个关键实践）

实践1：实体层纯POJO（最重要）

✗ 错误做法（传统分层架构）

```
@Entity
@Table(name = "orders")
public class Order {
    @Id
    private Long id;
    @Column
    private BigDecimal amount;
    @Enumerated
    private OrderStatus status;
    // JPA 注解污染了业务对象
}
```

✅ 正确做法（混合模式）

```
// domain/entity/Order.java - 纯 POJO, 无任何框架注解
public class Order {
    private Long id;
    private BigDecimal amount;
    private OrderStatus status;
    public void cancel() {
        if (status == OrderStatus.SHIPPED) {
            throw new IllegalStateException("已发货的订单不能取消");
        }
        this.status = OrderStatus.CANCELLED;
    }
    public boolean isOverdue() { /* 纯业务计算 */ }
}

// infrastructure/jpa/OrderPo.java - 数据库映射对象
@Entity
@Table(name = "orders")
public class OrderPo {
    @Id private Long id;
    private BigDecimal amount;
    private String status;
    public Order toDomain() { /* 转换逻辑 */ }
    public static OrderPo fromDomain(Order order) { /* 转换逻辑 */ }
}
```

收益：

- 业务规则可脱离数据库做单元测试
- 更换ORM框架只影响基础设施层
- 业务对象不被框架注解干扰

实践2：DTO隔离控制层与业务层

❌ 错误做法

```
@RestController
public class UserController {
    @GetMapping("/users/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.getById(id);
    }
}
```

✅ 正确做法

```
// controller/dto/UserResponse.java
public class UserResponse {
    private Long id; private String name; private String email; private String
    createdAt;
    public static UserResponse from(User user) {
        UserResponse resp = new UserResponse();
        resp.setId(user.getId());
        resp.setName(user.getName());
        resp.setEmail(user.getEmail().getValue());
        resp.setCreatedAt(user.getCreatedAt().format(DateTimeFormatter.ISO_DATE));
        return resp;
    }
}

// controller/UserController.java
@RestController
public class UserController {
    @GetMapping("/users/{id}")
    public UserResponse getUser(@PathVariable Long id) {
        User user = userService.getById(id);
        return UserResponse.from(user);
    }
}
```

收益：

- 前端不感知后端实体变化
- 可控制输出字段、隐藏敏感信息
- 实体修改不影响API契约

实践3：关键接口定义端口（按需使用）

✅ 需要定义接口的场景

```
// service/port/NotificationPort.java
public interface NotificationPort {
    void send(String userId, String message);
}

// infrastructure/sms/AliyunSmsAdapter.java
```

```
public class AliyunSmsAdapter implements NotificationPort { ... }
// infrastructure/sms/TencentSmsAdapter.java
public class TencentSmsAdapter implements NotificationPort { ... }
// infrastructure/mock/MockNotificationAdapter.java
public class MockNotificationAdapter implements NotificationPort { ... }
```

✗ 不需要定义接口的场景

```
@Service
public class UserService {
    @Autowired
    private UserRepositoryJpaImpl userRepository;
}
```

场景	是否需要接口	理由
数据库访问	✗ 不需要	极少替换
外部API调用	✓ 需要	可能切换供应商
缓存操作	⚠ 按需	替换可能性低
消息队列	✓ 需要	可能切换中间件
文件存储	✓ 需要	可能从本地换OSS

实践4：业务逻辑分层放置

- 通用规则 → 实体层
- 跨实体规则 → Service层
- 应用流程 → Service层

示例：订单取消

```
// domain/entity/Order.java
public class Order {
    public void cancel() {
        if (status == OrderStatus.SHIPPED) throw new IllegalStateException("已发货不能取消");
        this.status = OrderStatus.CANCELLED;
    }
}
// service/OrderService.java
@Service
public class OrderService {
    public void cancelOrder(Long orderId, String cancelReason) {
        Order order = orderRepository.findById(orderId);
        order.cancel();
        order.setCancelReason(cancelReason);
        orderRepository.save(order);
    }
}
```

```
        notificationPort.send(order.getUserId(), "订单已取消");
    }
}
```

四、完整代码结构示例

```
src/main/java/com/example/
├── controller/           # 表现层
│   ├── dto/
│   └── OrderController.java
├── service/             # 业务层
│   ├── port/
│   └── OrderService.java
├── domain/              # 实体层（纯POJO）
│   ├── entity/
│   └── valueobject/
├── repository/          # 数据访问层
└── infrastructure/     # 基础设施层
    ├── persistence/
    └── external/
```

五、4人小组分工建议

成员	负责模块	关注点
成员A	domain + controller/dto	实体设计、API契约
成员B	service核心逻辑	业务流程、端口接口
成员C	repository + infrastructure	数据访问、外部集成
成员D	测试 + 代码审查	架构规范执行

六、实施步骤（4周内落地）

阶段	时间	任务
第1周	3天	搭建基础结构
第1周	2天	团队培训
第2周	5天	核心功能开发
第3周	5天	按需引入端口接口
第4周	5天	审查、重构、测试

七、常见误区与纠正

误区	纠正
所有类都要有接口	只在可能替换的边界定义
实体必须完全无依赖	可依赖JDK标准类
每层都要单独DTO	Service/Repository可直接传实体
禁止使用框架注解	核心实体不用，其他层可用
严格遵守四层	可按复杂度合并简化

八、总结

混合模式 = 三层架构 + 实体纯POJO + DTO隔离 + 关键接口抽象

团队规模	推荐架构
2-3人	传统分层
4-6人	混合模式 
7-12人	完整整洁架构
12人以上	微服务+整洁

附录检查清单：

- domain 无任何框架注解
- Controller 出入参都是 DTO
- 外部调用通过 port 接口
- 核心业务在实体
- 无多余接口

成员C

Prompt

要求 AI 对比多种方案的优劣:请对比四种架构方案的优劣、适用场景，并说明是否适合本项目。

AI 回答

四种架构方案优劣对比（精简好懂版）

一、方案 A：分层架构（Layered）

优点：

- 结构清晰、简单稳定、易上手
- 无分布式、无异步，开发快、BUG少
- 维护成本低，适合小型项目

- 10周内可完成，风险最低

缺点：

- 扩展性一般，不能单独扩模块
- 耦合度高于微服务

适合：4人团队+10周校园项目 ✓

二、方案 B：MVC 架构

优点：

- 最简单、经典、学习成本低
- 页面+逻辑分离，开发速度最快
- 适合快速demo、校园小系统

缺点：

- 业务变大后Controller膨胀
- 复杂业务易乱，扩展性差

适合：小项目、界面交互多系统 ✓

三、方案 C：微服务架构（Microservices）

优点：

- 扩展性极强，可独立扩容、独立部署
- 模块高度解耦，大型系统稳定

缺点：

- 极复杂：网关、注册、分布式、联调麻烦
- 4人10周根本做不完
- 调试、排错、环境成本极高

适合：大厂高并发大型项目 ✗

四、方案 D：事件驱动架构（Event-Driven）

优点：

- 模块极度解耦
- 高并发、流量波动场景稳定

缺点：

- 异步难调试，流程难追踪
- 开发周期长，易数据不一致
- 10周来不及做稳

适合：高并发消息系统 ✗

最终结论

最优架构：分层架构（方案A） 理由：

1. 开发难度低、稳定
 2. 10周可完成
 3. 适合4人小团队
 4. 满足校园系统性能、维护、扩展需求
 5. 务实、可落地，符合课程项目要求
-

成员D

Prompt

让 AI 扮演架构师角色进行分析:请以专业架构师角度，评价四份方案并给出最终建议。

AI 回答

从专业架构师角度看：**4人团队、10周周期，应优先选择“分层架构 + MVC表现层”的单体架构。**

一、总体评价

四份方案对比合理：

- 分层架构：简单、清晰、无分布式问题
- MVC：开发快、学习成本低
- 微服务：强扩展但复杂度极高
- 事件驱动：强解耦但调试难

课程项目最怕选**超出团队能力**的架构，导致功能、测试、演示不稳。

二、方案专业评价

方案A：分层架构 —— 最适合做主架构

优势：

- 开发复杂度低
- 代码结构清晰、易分工
- 本地调用性能稳定
- 无需分布式、服务注册、消息一致性等高难度问题
- 10周可稳定交付核心功能

****不足：** **扩展性有限，只能整体扩容（课程项目可接受）。

方案B：MVC架构 —— 适合做表现层，不做主架构

MVC 是界面/请求组织模式，**不是完整系统架构**。更合理定位：

- 整体：分层架构

- 表现层：使用 MVC 思想

否则业务复杂后会出现**胖Controller**，维护困难。

方案C：微服务 —— 技术高级但不适合本项目

问题极多：

- 服务拆分、远程调用、网关、登录跨服务、分布式事务、部署、链路追踪、独立数据库..... 4人10周几乎不可能完成。 **建议：暂不采用，作为未来演进方向。**

方案D：事件驱动 —— 可局部借鉴，不做主架构

适合：通知、日志、消息、状态变更。 但全系统异步会导致：

- 事件模型设计复杂
- 消息可靠性、重试、幂等难处理
- 链路难调试 **建议：主体分层，非核心流程局部使用。**

三、最终架构建议

采用：

- 主体：分层单体架构
- 表现层：MVC思想
- 局部：事件驱动（通知/日志）
- 未来：微服务演进

四、对比表修正建议

MVC 不应与分层架构并列：

- 分层架构：系统整体架构
- MVC：表现层组织模式
- 微服务：分布式架构
- 事件驱动：通信解耦模式

五、架构师最终结论

最优选择：方案A 分层架构 辅助：方案B MVC 局部借鉴：方案D 事件驱动 暂不采用：方案C 微服务

团队辩论最终结论

最终选定架构

以 方案A：分层架构 为主体架构，结合 方案B：MVC 作为表现层实现方式

为什么选它？

1. 完全匹配 4人团队、10周周期 的现实约束

2. 开发难度低、结构清晰、风险最低
3. 可维护性、性能、交付可行性最优
4. 便于分工、开发、测试、答辩
5. 务实、可落地，是课程项目最合理选择

为什么不选其他方案？

- **不选 MVC**：只适合做界面层，不能作为完整系统架构，后期维护差
- **不选 微服务**：复杂度爆炸，4人10周不可能交付
- **不选 事件驱动**：异步调试难、周期长、短期风险高，不适合做主架构